

Munymo — Full Context Briefing for New Chat Session

Generated from the original build session — Jun 23, 2026

CRITICAL: What NOT to do

Before anything else — **do not rewrite working code**. The following are all fully built and working correctly. Read them before touching them:

- `client/src/pages/DailyGame.tsx` — the research metrics section (lines 640–784) renders a **two-column side-by-side comparison table** using `grid grid-cols-2`. Do not rewrite this.
 - `server/_core/scheduledCuration.ts` — the cron endpoint is fully built. Do not rewrite it.
 - `server/scoring.ts` — scoring logic is correct. Do not rewrite it.
 - `server/email.ts` — all email templates are built and tested. Do not rewrite them.
 - `client/src/components/ChartSheet.tsx` — the bottom-sheet chart panel. Do not rewrite it.
-

1. The Cron Job — FULLY BUILT, JUST NEEDS UNPAUSING

What it is

The daily curation is a **Manus Agent scheduled task** — NOT a server-side cron. It is a separate AI agent that runs on a schedule and calls the server's HTTP endpoint. The server listens; it does not schedule itself.

Current status

```
Task UID:      VKTf81x4Muj5Sx9c3JcCvi
Status:        active (was paused, re-enabled in previous session – check
Schedule:      Weekdays (Mon–Fri), 4:15 AM Perth time (AWST = UTC+8)
                = second 15300 of the day, SCHEDULE_TYPE_WEEKLY, days [1,2,3
Timezone:      Australia/Perth
Project UID:   SLgzupJDuKY6wiWMJF6c7P
runAsNewTask:  true
runMode:       ask_user
```

To check/fix the cron status

```
manus-config schedule status
```

If status is `pause` , re-enable it:

```
manus-config schedule update --uid VKTf81x4Muj5Sx9c3JcCvi --status active
```

What the agent does when it fires

1. GETs `https://munymo.com/api/scheduled/recent-games` — public endpoint, returns last 30 days of games
2. Looks up closing prices on Yahoo Finance for today's active game
3. Selects tomorrow's matchup (freshness rules: no same sector in 7 days, no same company in 30 days, no same pair in 365 days)
4. Researches tomorrow's companies, writes research content, metrics, validation question, Hindsight Spotlight Lesson
5. POSTs the complete JSON to `POST https://munymo.com/api/scheduled/daily-curation`
6. Server closes today's game, scores all picks, sends result emails, creates tomorrow's game

The agent prompt

The full prompt is at: `/home/ubuntu/munymo-mvp-fresh/references/daily-curation-agent-prompt.md`

Do not modify this file unless specifically asked.

The server endpoints (already working — do not touch)

Both are in `server/_core/scheduledCuration.ts` and mounted in `server/_core/index.ts`:

- `GET /api/scheduled/recent-games` — public, no auth required
- `POST /api/scheduled/daily-curation` — authenticated via session cookie

If the cron misses a run (manual fallback)

1. Open `references/daily-curation-agent-prompt.md`
2. Start a new Manus chat and paste the prompt, replacing `{RECENT_GAMES_JSON}` with the output of `GET https://munymo.com/api/scheduled/recent-games`
3. The agent will research and generate the JSON payload
4. Go to `munymo.com` → Admin → End of Day → Import JSON → paste the payload → Populate Fields → Publish & Create Next Game

2. Scoring System — Hardcoded Constants (Not Yet Wired to DB)

The scoring logic is in `server/scoring.ts`. Current hardcoded constants:

```
VALIDATION_FAST_THRESHOLD_MS = 15_000 // 15 seconds → full 20 pts
VALIDATION_SLOW_THRESHOLD_MS = 60_000 // 60 seconds → minimum pts
VALIDATION_MAX_SCORE = 20
VALIDATION_MIN_SCORE = 12
LEADERBOARD_QUALIFICATION_THRESHOLD = 20 // games needed to qualify
```

Prediction score is hardcoded at **80 points** for a correct pick, **0 for incorrect** (line 50 of `scoring.ts`).

The `AdminSettings` page (`/admin/settings`) saves scoring weights to the `app_settings` DB table, but `scoring.ts` **does not yet read from that table** — it still uses

the hardcoded constants above. This is a known outstanding item. The DB helpers `getSetting()` and `getAllSettings()` already exist in `server/db.ts` and are ready to use.

Outstanding task: Replace the hardcoded constants in `scoring.ts` with `await getSetting(key, defaultValue)` calls.

3. Email System — What Emails Are Sent and When

All email templates are in `server/email.ts`. Four templates exist:

Template	Subject	Trigger
<code>buildGameAvailableEmail</code>	“Today’s Munymo matchup is live — {A} vs {B}”	When admin activates a new game (creates it as active)
<code>buildResultPublishedEmail</code>	“Munymo result: {winner} wins — your score is {score}”	When End of Day runs and result is published
<code>buildMissedGameEmail</code>	“You missed it — {winner} beat {loser} on {date}”	Same End of Day run, for players who didn’t submit a pick
<code>buildStreakAtRiskEmail</code>	“Your {N}-day streak is at risk — play before lockout”	Not yet wired to a trigger (template exists, no automated send)

Email colours: brand green `#009050`, loss red `#c0392b`. Sent via Resend (`RESEND_API_KEY` env var).

4. Admin End of Day Flow

The `AdminEndOfDay.tsx` page (`/admin/end-of-day`) works as follows:

1. Admin pastes the curation JSON into the Import JSON textarea
2. Clicks **Populate Fields** — this parses the JSON and fills all form fields
3. A green **“Ready to publish”** summary card appears at the top of the page showing today’s closing game and tomorrow’s matchup

4. Admin clicks **Publish & Create Next Game** — this calls the `admin.endOfDay` tRPC procedure
5. A full-screen loading overlay with animated progress steps appears while the operation runs (locking picks → scoring → emails → creating game)
6. On success, the overlay shows a completion state

The **JSON payload format** is defined in `server/_core/scheduledCuration.ts` (the `CurationPayload` interface). Key fields:

- `today.winnerTicker`, `today.companyAPerf`, `today.companyBPerf`, `today.resultSummary`, `today.hindsightSpotlight`
- `tomorrow.gameDate`, `tomorrow.companyAName`, `tomorrow.companyATicker`, `tomorrow.companyBName`, `tomorrow.companyBTicker`, `tomorrow.sector`, `tomorrow.lockoutAt`, `tomorrow.researchContent`, `tomorrow.researchMetrics`, `tomorrow.validationQuestion`

researchMetrics format — this is critical for the side-by-side layout to work:

```
{
  "AAPL Market Cap": "$3.1T",
  "AAPL P/E Ratio": "32.4x (TTM)",
  "AAPL Revenue Growth": "+5.1% (YoY, FY2025)",
  "AAPL EPS (TTM)": "$6.42",
  "AAPL 52-Week Range": "$164.08 - $260.10",
  "AAPL Analyst Consensus": "Buy, avg target $237.45",
  "MSFT Market Cap": "$3.3T",
  "MSFT P/E Ratio": "35.2x (TTM)",
  ...
}
```

Each key **MUST** be prefixed with the ticker symbol followed by a space. The game page splits them by ticker prefix to build the two-column layout. If keys don't have ticker prefixes, the fallback splits the list in half.

5. Research Metrics Layout — How It Works

The two-column side-by-side layout is in `client/src/pages/DailyGame.tsx` lines 640–784.

The same layout is in `client/src/pages/ArchiveGame.tsx` — use that as a reference too.

Structure:

- Single rounded card with `border` and `background: var(--color-surface)`
- **Header row:** `grid grid-cols-2` — Company A (green `#009050`) left, Company B (blue `#1d4ed8`) right, ticker chip + company name
- **Metric rows:** `grid grid-cols-2` — alternating background shading, label in small uppercase, value in bold, `MetricExplanationSheet` “What does this mean?” link below each value
- **Chart CTAs row:** `grid grid-cols-2` at the bottom — “View Chart” button for each company, opens `ChartSheet` slide-up panel

The `ChartSheet` component is at `client/src/components/ChartSheet.tsx`. It renders as a fixed full-screen bottom sheet (z-index 200), uses `html.sheet-open` CSS class for scroll locking (NOT inline styles — this was a critical iOS bug fix).

6. Outstanding Items (from `todo.md` — genuinely not done)

High priority:

- **Wire scoring weights** from `app_settings` DB table into `server/scoring.ts` (small change)
- **Verify `/demo/autoplay`** on device — ghost cursor, tap-to-advance beats

Medium priority:

- **Push to GitHub** — sync latest to `pauljkwa/munymo-mvp-v1`
- **Add Resend DNS records** in Cloudflare (SPF/DKIM/DMARC from Resend dashboard)
- **Streak-at-risk email** — template exists but no automated trigger

Items marked as complete in `todo.md` but with `[ ] Push to GitHub` sub-items — these are just GitHub sync tasks, not functional gaps. The code is deployed and working.

7. Demo / Tutorial Mode

- `/demo` — static Apple vs Microsoft demo game, no auth required, no DB writes. File: `client/src/pages/DemoGame.tsx`

- `/demo/autoplay` — tap-to-advance animated walkthrough with ghost cursor. File: `client/src/pages/DemoAutoplay.tsx`
 - Voiceover script: `references/tutorial-voiceover-script.md` (v2, corrected)
 - The result commentary is called “**Hindsight Spotlight Lesson**” throughout the app — do not rename it
-

8. Design System Quick Reference

Token	Value	Use
<code>--color-brand</code>	<code>#002000</code> deep forest green	Buttons, nav active, headings
<code>--color-gold</code> (misnamed)	<code>#009050</code> emerald	CTAs, accent, Company A colour
<code>--color-success</code>	<code>#1a8a3a</code>	Correct answers, active badges
Company B colour	<code>#1d4ed8</code> blue	Hardcoded in research layout

Typography: Plus Jakarta Sans (body) + Syne (headings) via Google Fonts.

Logo: `MunymoLogo` component in `client/src/components/MunymoLogo.tsx`. Full wordmark PNG at `/manus-storage/munymo-logo-cropped_75fe3c86.png`.

9. Auth

Auth is **Clerk** (NOT Manus OAuth — earlier notes referencing Manus OAuth are stale).

- `VITE_CLERK_PUBLISHABLE_KEY` and `CLERK_SECRET_KEY` are set
 - `useAuth()` hook wraps Clerk’s `useUser()`
 - Server uses Clerk Express middleware for `protectedProcedure`
-

10. Key File Locations

<code>client/src/pages/DailyGame.tsx</code>	← Main game page (research metrics)
<code>client/src/pages/ArchiveGame.tsx</code>	← Archive game page (same metrics)
<code>client/src/pages/DemoGame.tsx</code>	← Static demo at /demo
<code>client/src/pages/DemoAutoplay.tsx</code>	← Animated walkthrough at /demo/a
<code>client/src/pages/admin/AdminEndOfDay.tsx</code>	← End of Day publish page
<code>client/src/pages/admin/AdminSettings.tsx</code>	← Scoring weights config
<code>client/src/pages/admin/AdminPlayers.tsx</code>	← Player management
<code>client/src/pages/admin/AdminDashboard.tsx</code>	← Admin overview
<code>client/src/components/ChartSheet.tsx</code>	← Bottom sheet for charts (iOS sc
<code>client/src/components/CandlestickChart.tsx</code>	← TradingView Lightweight Char
<code>server/routers.ts</code>	← All tRPC procedures
<code>server/db.ts</code>	← All DB query helpers
<code>server/email.ts</code>	← Email templates + send helpers
<code>server/push.ts</code>	← Web Push helpers
<code>server/scoring.ts</code>	← Score calculation (hardcoded co
<code>server/_core/scheduledCuration.ts</code>	← /api/scheduled/* endpoints
<code>references/daily-curation-agent-prompt.md</code>	← Cron agent prompt
<code>references/tutorial-voiceover-script.md</code>	← Tutorial voiceover script (v2)
<code>references/periodic-updates.md</code>	← How scheduled jobs work

End of briefing.